

ROOMREADY

Find your study space.

Study Room Reservation System

Engineering Residency POC • 5-hour Saturday sprint

June 13, 2026

Aaron Graf — Product Owner
Dr. Valerie Ann Haywood — Dev Lead
Aaron Tobias — Architect



The problem

Study room availability is unclear.

Patrons lose time checking rooms.
Staff lose focus answering routine availability questions.

What it costs

2

user groups

1

repeated question

**“Is there a study room
open right now?”**

Our solution

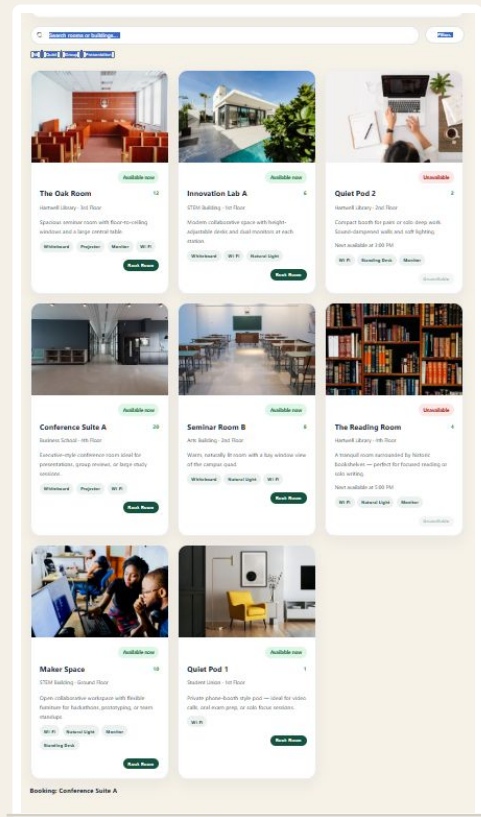
A kiosk-style frontend that lets patrons self-serve room reservations.

View availability → filter rooms → reserve → see status update

POC boundary

No authentication, payment, email notifications, production backend, or future scheduling.

Key features visible in the POC



01

Search + filter controls

Search rooms/buildings and filter by status.

02

Room cards

Image, room name, location, capacity, tags, and status.

03

Book Room action

Available rooms route into the booking form.

04

Reservation form

Name, date, time, email, and confirmation.

05

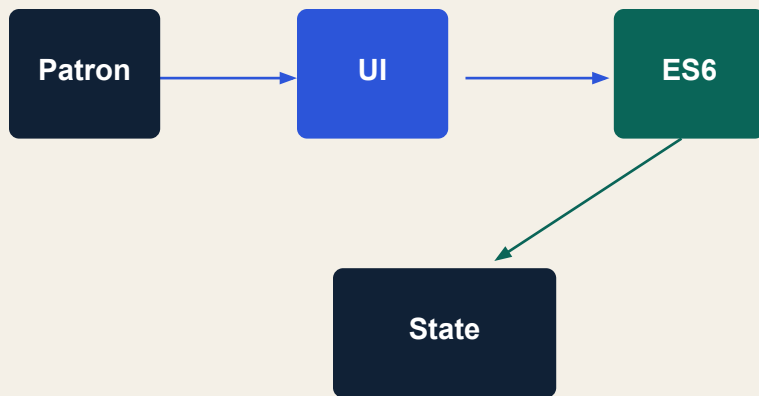
My Bookings + state update

localStorage supports a repeatable demo.

All five are visible in the current demo

Architecture overview

User → Web UI → JavaScript Logic → localStorage



Stack choice

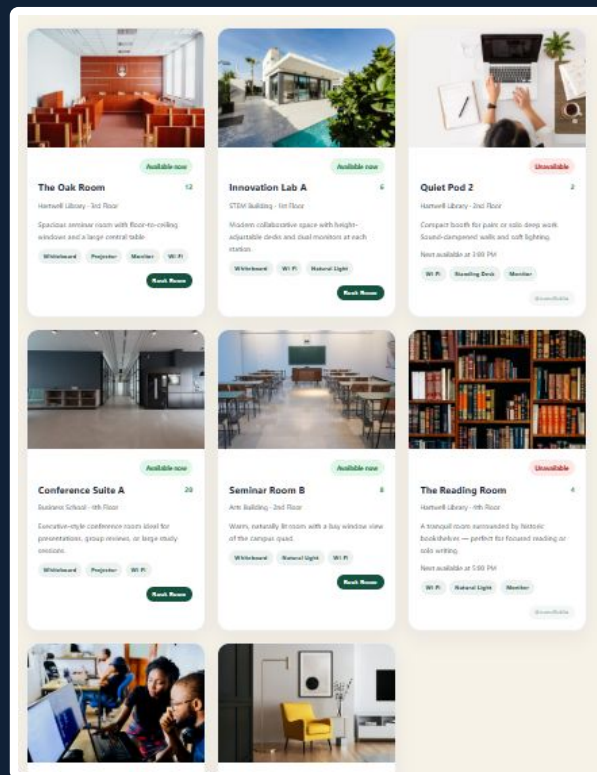
HTML5, CSS3, Vanilla JavaScript, and browser localStorage gave us a fast, lightweight, sprint-appropriate POC.

Live demo: discover a room

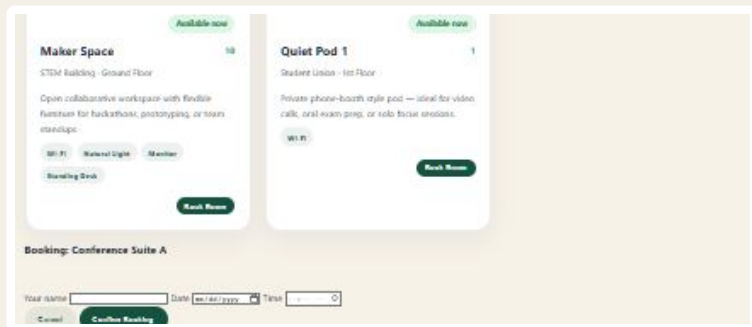
Narrate the patron's first question:

“What rooms are open right now?”

- 1 Start at the dashboard
- 2 Use the search bar or status filter
- 3 Point out room status labels
- 4 Select an available room



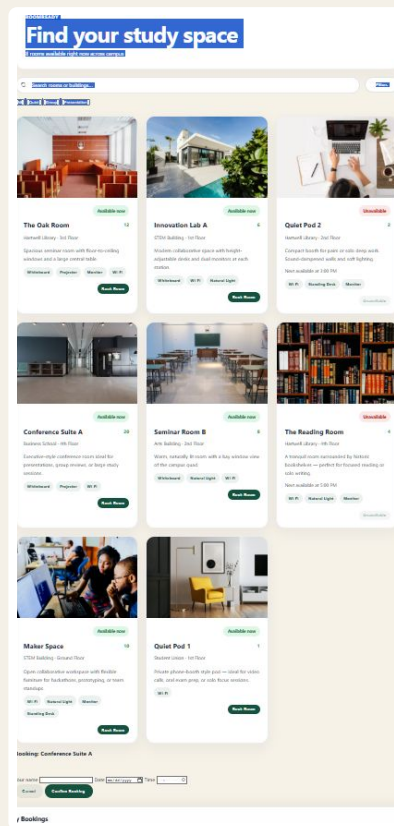
Live demo: reserve and confirm



- Selected room carries into the form
- Required details are captured
- Confirmation supports repeatable demo
- State is simulated, not production-grade

Booking closes the loop.

After submission, the reservation appears in My Bookings and the room status updates through localStorage.



Engineering artifacts

Evidence that the POC was scoped, documented, and built intentionally.

GitHub repo

Implementation files and project structure

Product backlog

12 stories / 34 points across 3 epics

SRS

Requirements tied to the visible POC flow

UML / ADR

Architecture and decision rationale



Problem → Backlog → Requirements → POC → Demo

Decision and trade-off

Use simulated frontend data instead of a backend.

Protected the sprint

- Proved the user flow first
- Reduced implementation risk
- Kept the demo repeatable

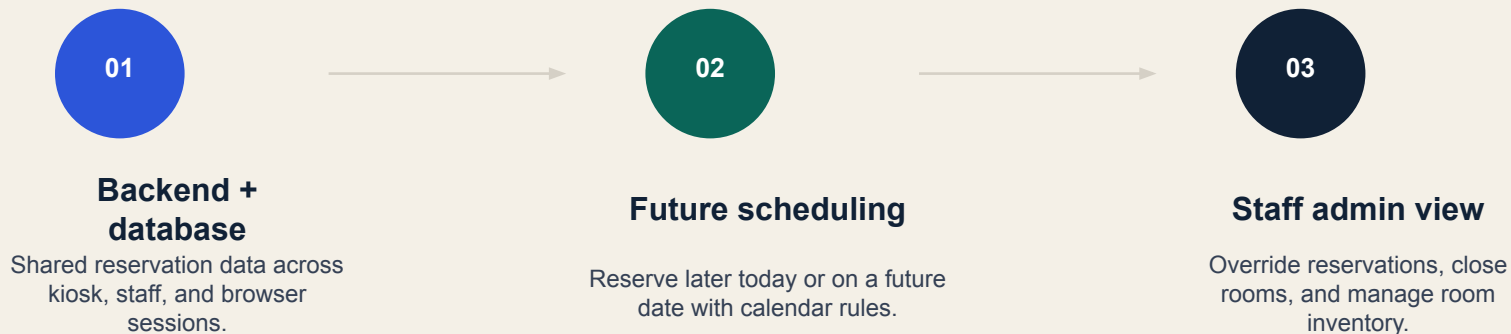
Deferred production concerns

- Shared database state
- Server-side validation
- Concurrency and security

Frontend simulation first → backend integration next

What comes next

If RoomReady became a 16-week project:



Questions?

RoomReady: Study Room Reservation System

Faculty Q&A focus: user flow, architecture, scope, trade-offs, and next steps.

